

AI융복합연구단 제2회 월례세미나

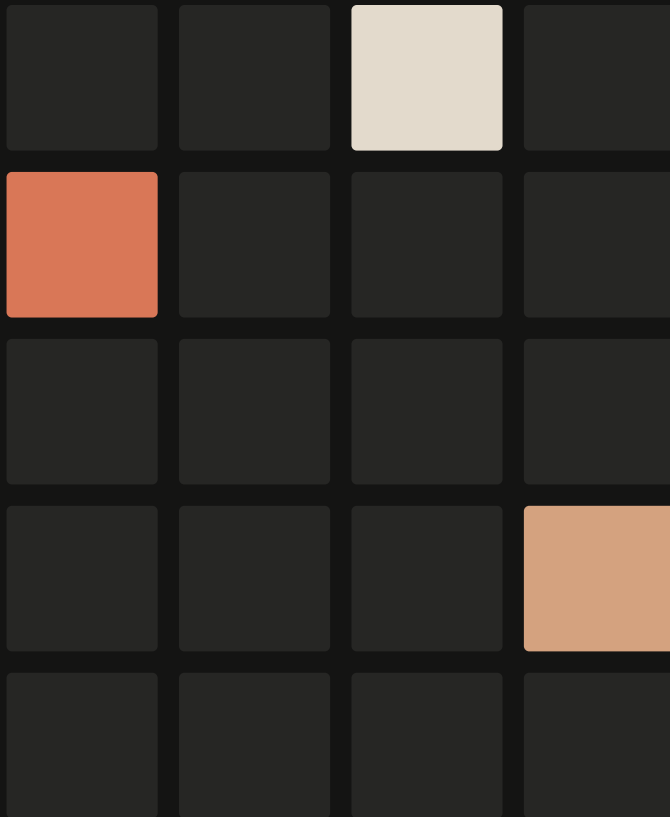
AI 도구 활용 실습!

Claude Code 시작하기

백 동 해

한국환경연구원 환경평가본부

2026. 08. 27.



Section 1

왜 Chat이 아니라 Agent인가?

LLM 활용 패러다임의 진화 · 에이전트(agent) · 하네스(harness)

LLM 활용 패러다임의 진화

모델의 성능이 아니라, 모델을 둘러싼 시스템이 결과를 결정한다

1단계 프롬프트 엔지니어링(prompt engineering) 2023~

핵심질문: “모델에게 어떻게 물어볼 것인가” — Few-shot, Persona prompting 등 질문(프롬프트) 작성 기법

한계점: 잘 지시해도 모델이 모르는 정보는 답하지 못한다

2단계 컨텍스트 엔지니어링(context engineering) 2024~2025

“모델에게 무엇을 보여줄 것인가” — RAG, 메모리, 컨텍스트 압축

한계점: 필요한 정보를 쥐도 복잡한 작업을 끝까지 실행·검증하지 못할 수 있다.

3단계 하네스 엔지니어링(harness engineering) 2026~

“실행 환경을 어떻게 설계할 것인가” — Agentic System = Model + Context + Tools + Control Loop + Environment + Feedback

한계점: 다중 또는 연속적인 작업이 가능해지면서 비용·보안·운영이 문제가 된다.

Prompt는 지시를 설계하고, Context는 판단에 필요한 정보를 설계하며, Harness는 그 판단을 실제 작업으로 연결하는 시스템을 설계한다.

하네스 엔지니어링이란 무엇인가

OpenAI 엔지니어링 블로그 「Harness engineering — agent-first world」(2026.02.11) 요약

정의 — 하네스(harness)

과제를 이해하고, 맥락을 유지하고, 도구를 부르고, 실패를 처리하고, 승인을 요청하며 작업을 이어가는 — 모델을 둘러싼 그 실행 시스템 전체가 하네스다.

다섯 달의 실험에서 정리한 원칙 — 엔지니어 3명, 손으로 쓴 코드 0줄, 약 100만 줄

저장소가 곧 기록 시스템이다

지침서를 크게 쓰지 말고 ‘지도’를 준다.
AGENTS.md·CLAUDE.md는 목차여야 한다.

규칙은 기계로 강제한다

린터·구조 테스트로 불변식을 지키고,
구현 방식은 에이전트에게 열어 둔다.

에이전트가 읽을 수 있게 만든다

실행 중 맥락에 넣지 못하는 지식은 없는 것과 같다 — 채팅·문서·사람 머릿속의 것.

실패하면 환경을 고친다

‘더 시켜본다’가 아니라 ‘무슨 역량이 빠졌는가’를 묻고 저장소에 되먹인다.

사람이 조정하고, 에이전트가 수행 — 환경 설계가 본업이 된다

하네스 엔지니어링: 에이전트 우선 세계에서 Codex 활용하기

작성자: Ryan Loppolo, 기술 스태프 멤버

문서 내용 보기 22:05

공유

지난 5개월 동안 우리 팀은 **수동으로 코드 라인을 전혀 작성하지 않고** 소프트웨어 제품의 내부 배타를 구축하고 출시하는 실험을 진행해 왔습니다.

이 제품에는 내부 사용자와 외부 알파 테스트가 배포되어 있고 출시, 배포, 고장, 수리 작업이 진행됩니다. 다른 많은 애플리케이션 로직, 테스트, CI 구성, 문서화, 관측 가능성, 내부 통합 등 모든 코드 라인이 Codex에 의해 작성되었다는 것입니다. 수작업으로 코드를 작성했을 때 걸렸을 시간의 약 1/10 정도의 시간으로 작성된 것으로 추정됩니다.

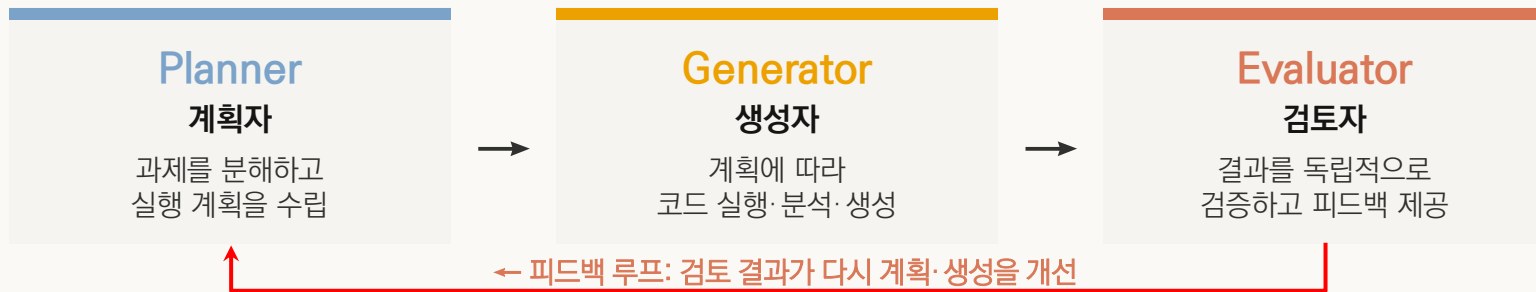
사람이 조정하고, 에이전트가 수행

엔지니어링 속도를 몇 배로 높이는 데 필요한 것을 구축하기 위해 의도적으로 이 제약을 선택했습니다. 결국 백만 개 라인에 달하는 코드를 출시하는 데 몇 주가 걸렸습니다. 이를 위해 우리는 소프트웨어 엔지니어링 팀의 주된 임무가 더 이상 코드 작성에서 벗어나 환경을 설계하고, 의도를 명시하며, Codex 에이전트가 안정적인 작업을 수행할 수 있도록 피드백 루프를 구축하는 것으로 바뀔 때 어떤 변화가 발생했는지 이해해야 했습니다.

이 글에서는 에이전트 팀과 함께 완전히 새로운 제품을 구축하면서 얻은 교훈, 즉 무엇이 문제이고 무엇이 보완되어야 하는지, 그리고 정말 부족한 자원의 시간과 주의를 극대화하는 방법에 대해 설명합니다.

하네스 엔지니어링 예시

모델은 지능을 제공하고, 하네스가 그 지능을 실제 작업으로 바꾼다



Planner

계획자

Claude Code에서는
Plan Mode로 작업을 분해
하고, CLAUDE.md로 프로
젝트 맥락을 전달한다.



Generator

생성자

Claude Code에서는
터미널과 파일 읽기·쓰기
로 코드를 실제로 실행
하고 결과를 본다.



Evaluator

검토자

Claude Code에서는
테스트를 돌려 확인하고
에러가 나면 스스로 고쳐
다시 시도한다.

기존 웹 챗봇 기반 수동 작업 예시

예시 상황 — 방대한 양의 수질관측자료가 주어진 상태에서, 이 자료를 이용하여 수질예측 딥러닝 모델을 개발하는 경우,

+

xx수질관측망에서 수집된 10년간의 수질데이터가 있다. 이 자료를 학습자료로 활용해서 딥러닝 모델을 python으로 작성하고 결과를 분석해줘



1	데이터 파악	파일 구조를 사람이 텍스트로 설명해야 한다 → 데이터의 통계 및 특성에 대한 설명 누락	수동
2	코드 작성	코드 복붙 → 에러 → 에러 복붙 → 에러... (언제까지? 뭘때까지!)	복붙 루프
3	탐색적 분석	결과를 직접 보지 못한다 → 연구자가 판단을 중개	맥락 단절
4	장기간 작업	새 대화를 열면 전과정 내용을 처음부터 다시 설명	맥락 유실
5	결과 정리	코드·그래프·데이터·해석이 모두 흩어져 있다	분산

~30회

복붙 횟수

15~25회

에러 루프

3단계부터

맥락 유실

50% 이상

중개에 쓴 시간

낮음

재현성

하네스 기반 Agent 도입 시

Planner — 자동 계획 수립

1. 데이터 파악: ./data/ CSV 구조 분석
2. 전처리: 결측치, 이상치, 시간 통일
3. 탐색적 분석: 시계열 패턴, 계절성
4. 모델 개발: ARIMA → LSTM 비교
5. 평가: RMSE, MAE, 예측 그래프
6. 결과 정리: 보고서 + Figure 생성

Generator + Evaluator — 자동 순환 실행

단계	Generator (실행)	Evaluator (검증)
데이터 파악	ls, head, describe() 실행 → 구조 파악	컬럼 타입, 결측치를 자동 검증
전처리	pandas 스크립트 작성·실행	전후 행 수 비교, 결측치 0 확인
모델 개발	예측모델(딥러닝 등) 코드 작성·학습	과적합 여부, train/val loss 검증
결과 정리	보고서 + Figure 일괄 저장	파일 목록, 평가 지표 최종 확인

에러 자동 수정: ImportError → pip install 자동 실행 → 재시도 → 성공 → 다음 단계

① 계획 수립

작업 수행 전 작업절차를 사전에 계획하고
작업 도중 문제가 발견될 경우 스스로 보완하여 계획 수정

② 실행

로컬 PC환경에서 직접 코드를 작성하고,
실행결과를 스스로 확인

③ 검증

결측치 확인, train/val loss 등 모델의 적정성을
스스로 점검한다. (검보정 자동화)

④ 정리

보고서와 Figure를 프로젝트 폴더에 파일로 남긴다

웹 CHAT vs AGENT

항목	웹 CHAT	AGENT
데이터 접근	연구자가 구조를 설명해 전달	직접 파일을 읽고 구조를 파악
코드 실행	복붙 → 에러 → 복붙 루프	직접 실행하고 에러를 자동 수정
맥락 유지	대화가 길어지면 유실된다	프로젝트 폴더 전체가 맥락이다
계획 수립	연구자가 단계별로 수동 지시	Planner가 자동으로 분해한다
결과 검증	연구자가 읽고 판단해 전달	Evaluator가 자동 검증·피드백
산출물	대화 속에 흩어진 코드 조각	폴더에 정리된 파일 구조
재현성	낮다 — 다시 재현하기 어렵다	높다 — 스크립트를 보존하고 재실행한다

같은 임무, 같은 모델 지능 — 하네스 기반 AGENT가 있으면 연구 보조원, 없으면 번역기다

하네스 기반 Agent를 실현하려면 무엇이 필요한가

세 가지 능력이 모두 있어야 에이전트가 성립한다. 웹 챗봇에는 셋 다 없다.



① 파일시스템 접근

프로젝트 코드와 데이터
파일을 직접 읽고 쓴다.

웹 챗봇 복붙으로 전달
CLI 도구 직접 읽고 쓴다



② 코드 직접 실행

분석 스크립트를 쓰고
바로 실행해 에러까지
고친다.

웹 챗봇 샌드박스에 갇힘
CLI 도구 내 로컬에서 실행



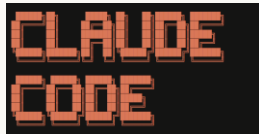
③ 프로젝트 맥락

수천 줄 코드와 수십 개
설정 파일을 이해한 채
작업한다.

웹 챗봇 대화창이 한계
CLI 도구 폴더 전체 접근

이 셋을 구현하려면 운영체제 수준의 접근이 필요하다. 그래서 CLI 기반의 Agent이어야 한다.

주요 Agent(CLI기반) 도구



항목	Claude Code	Codex	Antigravity
개발사	Anthropic	OpenAI	Google
기반 모델	Claude Opus 5	GPT-5 계열 Codex	Gemini 3.5 Pro
프로젝트 지침파일	CLAUDE.md	AGENTS.md	AGENTS.md
컨텍스트 크기	기본값 200K (확장: 1M)	1M	258K (공식: 400K)
요금	유료 구독	유료 구독	무료 티어 있음

무료 플랜으로는 Claude Code를 쓸 수 없다 — Pro 이상이 필요하다

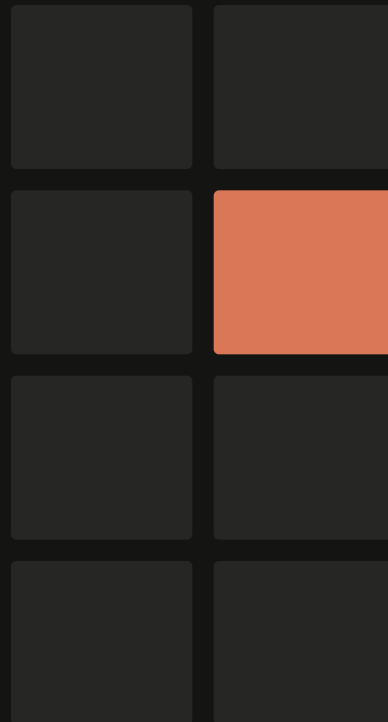
공식 문서 원문 — code.claude.com/docs/en/setup

Claude Code requires a Pro, Max, Team, Enterprise, or Console account.
The free Claude.ai plan does not include Claude Code access.

Section 2

시작하기

설치 · 셋업 · Skills



Appx. Claude app: Claude Code VS Cowork

둘 다 같은 Claude 모델을 쓴다. 다른 것은 일하는 방식이다. Cowork·Code는 유료 플랜 전용

Claude Code

코드를 쓰고 실행하고
고친다.

이럴 때

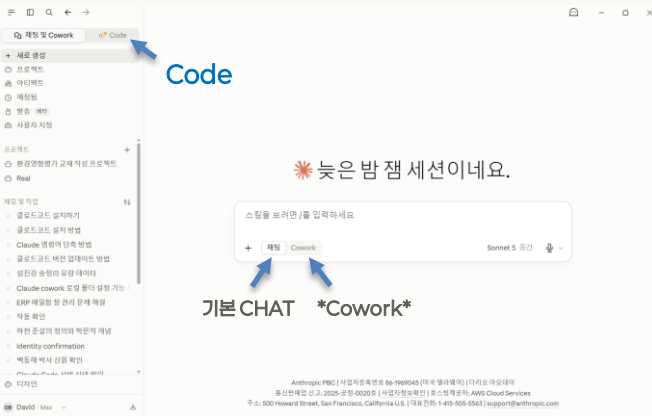
- 데이터 분석 스크립트
연구용 도구 만들기

Claude Cowork

목표만 말하면 내 파일과
도구를 오가며 끝낸다.

이럴 때

- 폴더 자료로 문서 작성
- 엑셀 정리·대조·검산 등



항목	Claude Code	Cowork
실행위치	로컬PC(내 PC)	클라우드 샌드박스
대상	개발자, 엔지니어	일반 연구자, 사무직
효율	압도적. grep/find는 0.1초	같은 작업에 훨씬 많은 토큰 소모
진입장벽	터미널 환경	없음

Appx. Claude Cwork?

활용 예시

예시 상황

섬진강 수위-유량관측소 자료 10년치를 다운로드하여 그래프를 작성하고, 유황분석 등 기초적인 통계 분석

Cowork로 작업하면 무엇이 해결되는가? (그냥 하면 안되나...)

- 데이터 확보 : 국가 수환경 관측망 자료는 일반적으로 한번에 다운로드 할 수 있는 시간범위가 한정적 (한번 1개월치) – 다운로드노가다필요
- 데이터 포맷 : 제공되는 파일의 자료형식이 데이터 분석에 용이하지 않음 – 전처리 필요 (아긔역사노가다)
- 데이터 파편 : 위의 "데이터 확보"의 문제와 연계된 문제, 다수의 파일을 별도 포맷으로 다시 통합해야 함 (이것도...)

반복적인 귀찮은 작업을 한방에 해결! 다수의 분산된 파일을 내 PC에서 직접 처리!

Cowork 프롬프트 예시

[#](https://www.wamis.go.kr/wkw/flw/flw_dtdata.do?code=4009630&name=%25EA%25B5%25AC%25EB%25A1%2580%25EA%25B5%25B0%25EC%2586%25A1%25EC%25A0%2595%25EB%25A6%25AC)
이 사이트는 wamis의 섬진강 송정리의 유량 관측결과이다.
과거 10년간의 실시간 일자료 유량을 각각 다운로드해서 저장해줘.
chrome 익스텐션을 이용.

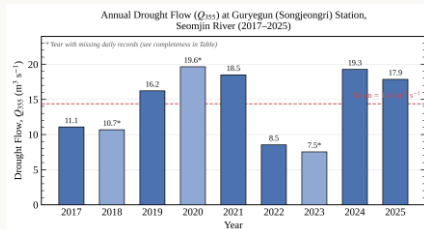
+ 채팅 Cowork

Sonnet 5 중간



프로젝트 수동

year	n_total	n_valid	completen	ess_%	Q95_abun	Q185_nor	Q275_low	Q355_dro	annual_m	annual_m
2017	365	365	100	28.23	19.6	14.21	11.06	30.45	6.45	
2018	365	350	95.9	44.04	28.03	18.42	10.66	67.35	9.28	
2019	365	365	100	40.44	25.06	19.43	16.21	63.44	11.11	
2020	366	350	95.6	49.18	28.88	23.06	19.63	132.53	17.68	
2021	365	365	100	38.72	26.21	21.49	18.48	49.09	15.17	
2022	365	365	100	21.39	16.33	12.05	8.53	25.89	6.51	
2023	365	341	93.4	59.83	22.63	12.4	7.52	121.05	6.75	
2024	366	366	100	62.78	35.71	28.6	19.26	76.48	14.59	
2025	365	365	100	64.99	28.93	21.67	17.85	92.34	17.29	
2026	238	238	100	33.1	20.45	16.51	11.21	31.62	9.69	



* 본 예제는 Chrome과 Claude for Chrome 사전 설치 필요

설치와 기본 셋업

설치 · 셋업

터미널에 그대로 입력

```
# 설치 - Windows PowerShell. Node.js 필요 없음
$ irm https://claude.ai/install.ps1 | iex

# 분석할 폴더로 이동해서 실행
$ cd D:\내연구\수온분석
$ claude (또는 claude --dangerously-skip-permissions)

# 첫 실행 후 한 번만 - 프로젝트 맥락 생성
> /init

# 자주 쓰는 명령
> /help /model /compact /plan /effort
```

macOS · Linux · WSL 은 `curl -fsSL https://claude.ai/install.sh | bash`

① 설치

Node.js는 따로 필요 없다. 명령어 한줄!
확인 — `claude --version`

② 첫 실행 · 로그인

이동한 폴더가 작업 범위가 된다.
Pro·Max 구독 필요 — 무료 플랜 불가

③ /init

CLAUDE.md가 만들어진다.
프로젝트 설명서이자 맥락이 된다

왜 CLAUDE.md를 만들어야 하는가

프로젝트 맥락 파일

프로젝트 폴더의 CLAUDE.md — 실제로 이런 것을 적는다

데이터

- 관측 CSV의 결측치는 -9999다. 평균 전에 NaN으로 바꿀 것
- 시간대는 KST다. UTC로 바꿀 때는 반드시 그 사실을 적을 것

코드

- 파일 입출력은 encoding='utf-8' 명시 (없으면 한글 깨짐)
- 그림 폰트는 Malgun Gothic, 음수 기호 깨짐 방지 설정

결과물

- 표는 xlsx, 그림은 300dpi PNG로 dist/ 아래에 저장
- 보고서 문체는 '~하였다' 체. 개조식으로 쓰지 말 것

- ① 매번 처음부터 시작한다
어제 알려준 것을 오늘은 모른다.
그 설명을 파일로 고정해 둔 것
- ② 코드에 없는 것은 추측한다
결측치 -9999, 시간대 KST —
모르면 지어내고, 지어내면 틀린다
- ③ 같은 함정을 두 번 안 밟는다
틀렸을 때 한 줄 덧붙이면
다음 세션부터 그 실수가 없다 (실수 발생시 보완 업데이트)
- ④ 사람이 읽는 문서가 된다
그냥 마크다운 파일이다.
새로운 세션에서도 동일한 환경으로 작업.

프롬프트를 더 잘 쓰는 것보다, 매번 필요한 맥락을 파일 하나에 적어 두는 편이 결과를 더 크게 바꾼다.

200줄 이내를 목표로 한다 — 매 요청마다 전부 들어가므로 길면 준수율이 떨어진다

자주 쓰는 슬래시 커맨드

전체 90여 개 슬래시 커맨드 가운데 비개발 연구자에게 실제로 쓸모 있는 것만 골랐다

/help

명령 목록을 보여준다

/clear

새 대화를 시작한다 — 작업이 바뀌면 무조건

/init

이 폴더용 CLAUDE.md 를 자동으로 만든다

/memory

CLAUDE.md 편집과 자동 메모리(memory) 관리

/context

지금 컨텍스트(context)에 무엇이 들어 있는지 그림으로

/btw <질문>

대화 기록에 남기지 않는 겉가지 질문 — 컨텍스트 오염 방지

/model

작업 유형 및 난이도별 모델 선택 (Opus, Sonnet 등)

/effort

각 모델별 추론을 위한 노력 수준 설정

/doctor

설정 전체를 점검하고 자동으로 고친다

/plan

계획 모드로 들어간다 — 바꾸기 전에 파악

/resume

예전 대화를 이어서 한다

/insights

최근 내 세션들을 분석해 HTML 리포트를 만든다

“내가 AI 를 어떻게 쓰고 있나” 자기점검

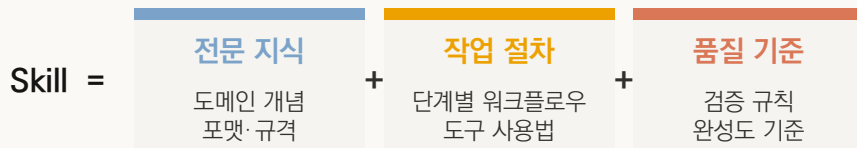
작업이 바뀌면 무조건 **/clear** — 이 열 개 중 가장 자주 쓰게 될 명령이다

Skills — 작업 절차를 통째로 재사용한다

Claude Code에 전문 지식을 주입하는 재사용 가능한 확장 모듈(skill)

Skill이란?

Claude Code에 전문 지식을 주입하는 재사용 가능한 확장 모듈



비유:

연구실의 실험 프로토콜과 같다.
한 번 작성하면, 누구든 같은 품질로 작업을 수행할 수 있다.

문서 생성/편집

/pptx /hwp /docx /pdf

오케스트레이션(협업)

/superpowers /gstack
/skill-codex

작동 방식

/pptx 발표자료를 만들어줘

①

지식 주입

Skill의 전문 지식이
Claude의 컨텍스트에 로드



②

워크플로우 설정

도구 사용법과
작업 순서가 활성화



③

품질 기준 적용

검증 규칙으로
결과물 품질 보장

디자인

/frontend-design

직접 제작

/skill-creator

* /plugin (marketplace) : 공개된 skill 목록 조회 및 설치

예시 — /pptx Skill이 실제로 하는 일

Skill 동작 예시

/pptx 이 논문을 읽고 발표자료를 만들어줘

① 전문 지식 주입

SKILL.md(디자인 가이드) + pptxgenjs.md(API 사용법) + 11개 스크립트 + 45개 XML 스키마 로드

② Planner — 설계

디자인 시스템(색상·폰트·레이아웃) 확인 → 슬라이드 구조 설계 → 콘텐츠 배치 계획

③ Generator — 생성

PptxGenJS 스크립트 작성 → node 실행 → .pptx 파일 생성

④ Evaluator — 검증

PPTX → PDF → PNG 변환 → 시각적 QA 수행 → 문제 발견 시 ③으로 되돌아가 수정

/pptx Skill 구성

문서 3개

SKILL.md
editing.md
pptxgenjs.md

스크립트 11개

unpack, pack,
thumbnail, validate
add_slide, clean ...

XML 스키마 45개

PPTX 포맷
검증용 규격 파일

이 절차가 Planner → Generator → Evaluator 순환 루프의 실제 구현의 예

Level 1

읽고 쓰기

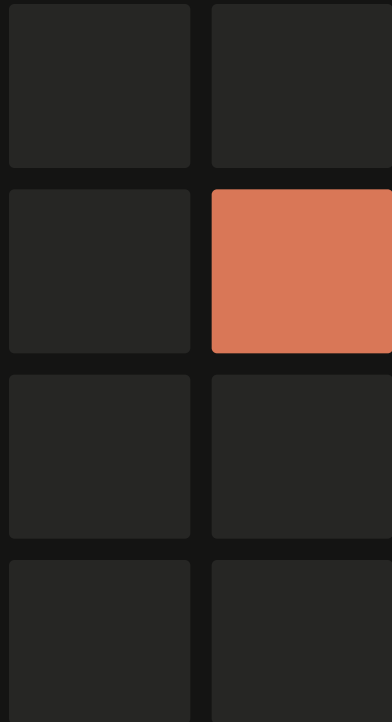
연구 문헌 · 발표자료 · 보고서

Level 1
읽고 쓰기

Level 2
같이 생각하기

Level 3
코드 실행

Level 4
도구 만들기



Demo ① 논문 PDF에서 학술대회 발표자료까지

Demo ①

입력 프롬프트 (1) — 규칙부터 만든다

> 이 프로젝트의 SCI 논문(PDF)을 분석해서 AGU 국제학술대회에서 발표할 ppt를 제작할 것이다. 아래 사항을 고려해서 CLAUDE.md 파일을 생성해줘.

- 15분 정도 구두발표 분량
- 청중은 관련 분야 박사급 이상의 전문가
- AGU 구두발표자료 작성 지침 확인
- 그림·도표는 PDF를 crop해서 사용
- 공학 스타일의 발표자료

① 규칙부터 만든다

목적·청중·분량을 CLAUDE.md에 먼저 적어 둔다

② 개요 설계

논문을 읽고 outline을 md 파일로 저장한다

③ 섹션별 세부 계획

한 번에 다 하지 않는다. 섹션 1부터 채운다

④ 검수

섹션 간 논리와 논문과의 일치를 다시 확인한다

한 번에 “PPT 만들어줘”가 아니다. 다음 장에 실제로 입력한 프롬프트가 순서대로 있다.

Demo ① 실제로 입력한 프롬프트

Demo ①

입력 프롬프트 (2)~(6) — 순서대로 입력했다

(2) 발표자료 개요 설계

> PDF를 분석해서 발표자료 개요(outline)를 설계하고 presentation_outline.md로 저장.

(3) 섹션별 세부 내용 계획

> 각 섹션별 구체적인 내용을 작성해서 @presentation_outline.md 업데이트. 섹션 1만 먼저.

(4) 논리적 연계성 검수

> 각 섹션 내용 간 논리적 연계성을 검토하고, SCI 논문 내용과 일치하는지 검증해줘.

(5) PPT 디자인 철학 설계

> 논문 내용·분야·학회 특성을 고려해 디자인 철학을 설계하고 design_philosophy.md에 저장.

(6) PPT 작성과 시각 검수

> outline을 design_philosophy 기준으로 작성. 스크린샷을 찍어 시각적 문제를 확인.

단계마다 계획하고 검수한다. 결과가 부족하면 피드백으로 개선을 요청한다

Demo ② 논문 · 보고서 작성

Demo ②

① 개요 설계

프로젝트 내 주어진 자료를 기반으로 챕터(목차)부터 설계

- 참고문헌, 주요연구 결과, 데이터, 모델링 결과, 기타 가용한 모든 자료 폴더에 수록
- 챕터(목차)부터 만들고 사람이 검수하여 논의 통해 최종 목차 결정

② 소챕터로 구체화

각 챕터를 더 작은 단위로 분할하여 소챕터 수준까지 계획한다

- 각 소챕터별 주요내용을 간단히 요약 (소챕터에 포함될 소주제 또는 주요내용)

③ 내용 작성

각 소챕터 레벨에서 주요내용별로 문단 단위로 작성

- 한번에 전체 내용 작성 지양

④ 연계성·사실 검증

개별적으로 작성된 챕터별 내용에 대해 논리적 연계, 가독성, 인용의 사실 여부를 검증

- 논리적 공백, 내용의 오류, 문단 간 연결성을 고려하여 반복적으로 시와 논의.

논문 및 보고서 작성시에는 엄격한 인간 감독하에 문단별 승인 과정을 통해 사실여부를 확인하는게 중요

* 컨셉, 초안, 브레인스토밍 수준에서 사용할 것을 권고

Level 2

같이 생각하기

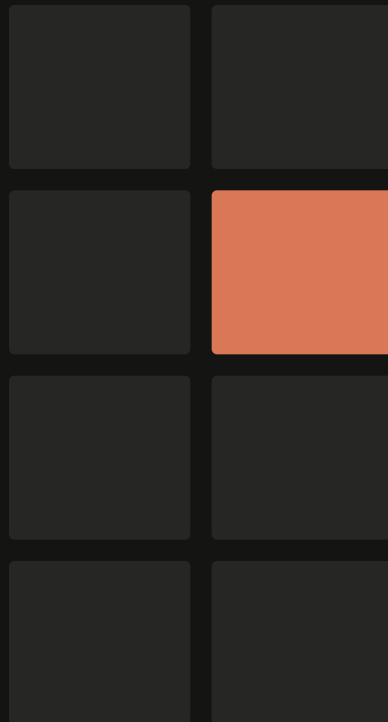
연구 전략 · 의사결정 · 방법론 설계

Level 1
읽고 쓰기

Level 2
같이 생각하기

Level 3
코드 실행

Level 4
도구 만들기



/deep-research 가 하는 일

공식 번들 워크플로(workflow) — 여러 서브에이전트(subagent)를 부린다

실행 명령

```
> /deep-research AI 데이터센터 건설 시 다양한 환경매체에  
미치는 영향과 최근 국내 AI 데이터 건설 동향에 대해서 조사해줘
```

```
# 백그라운드로 돌아간다.
```

```
# 조사가 도는 동안 세션은 계속 쓴다.
```

```
# 스킬보다 상위 개념인 '워크플로'다.
```

```
# 여러 서브에이전트를 오케스트레이션한다.
```

■ 스크립트를 직접 쓴다

JavaScript 오케스트레이션 코드

■ 병렬 서브에이전트 조사

여러 각도로 나눠 각자 웹 검색

■ 교차검증하고 투표한다

주장마다 서로 대조한다

■ 미통과 주장은 폐기

최종 보고서에서 걸러진다

■ 인용 달린 보고서

구조화된 문서로 나온다

나를 심문하는 스킬 — grill-me

<https://github.com/mattpocock/skills/commits?author=mattpocock>

스킬 사례 · 코드를 짜지 않는 스킬

한 번에 한 가지씩 — 답이 다음 질문을 정한다

```
$ /grill-me  
> 하천 수질 자료로 예측 모델을 만들어 보고 싶어
```

```
# 질문 1 — 무엇을 예측할 것인가  
다음 달 평균 BOD 농도인가요, 아니면 기준을  
초과하는 날인가요?  
권장: 초과일. 담당자가 실제로 쓰는 판단입니다
```

```
> 초과일로. 근데 기준을 월로 잡을지 모르겠어
```

```
# 질문 2 — 앞의 답에서 갈라져 나온 가지
```

아이디어가 아직 흐릴 때 — 질문을 어떻게 해야 될지 모를 때

- ① 한 번에 한 질문만
질문을 쏟아붓지 않는다.
내 답이 다음 질문을 만든다
- ② 추천 답을 같이 준다
"저라면 이렇게"까지 붙는다.
고르거나, 반박하면 된다
- ③ 사실은 스스로 찾는다
파일을 보면 알 수 있는 건
묻지 않고 직접 확인한다
- ④ 결정이 남을 때까지
설계의 갈래를 하나씩 닫는다.
다 닫히면 끝난다

좋은 답을 받는 법이 아니라 좋은 질문을 받는 법이다. 스킬 본문은 마크다운 여섯 줄이 전부다.

만약 Grill-me 스킬이 없다면? Claude code에서 시킨다! ("`>grill-me` 스킬 설치해줘!")

LLM 위키 — LLM이 쓰고 관리하는 지식베이스

안드레이 카파시(Andrej Karpathy)가 2026년 4월 공개한 활용 패턴

LLM Wiki

A pattern for building personal knowledge bases using LLMs.

This is an idea file, it is designed to be copy pasted to your own LLM Agent (e.g. OpenAI Codex, Claude Code, OpenCode / F1, or etc.). Its goal is to communicate the high level idea, but your agent will build out the specifics in collaboration with you.

The core idea

Most people's experience with LLMs and documents looks like RAG: you upload a collection of files, the LLM retrieves relevant chunks at query time, and generates an answer. This works, but the LLM is rediscovering knowledge from scratch on every question. There's no accumulation. Ask a subtle question that requires synthesizing five documents, and the LLM has to find and piece together the relevant fragments every time. Nothing is built up. NotebookLM, ChatGPT file uploads, and more RAG systems work this way.

The idea here is different. Instead of just retrieving from raw documents at query time, the LLM **incrementally builds and maintains a persistent wiki** — a structured, interlinked collection of markdown files that sit between you and the raw sources. When you add a new source, the LLM doesn't just index it for later retrieval. It reads it, extracts the key information, and integrates it into the existing wiki — updating entity pages, revising topic summaries, noting where new data contradicts old claims, strengthening or challenging the evolving synthesis. The knowledge is compiled once and then kept current, not re-derived on every query.

This is the key difference: the wiki is a **persistent, compounding artifact**. The cross-references are already there. The contradictions have already been flagged. The synthesis already reflects everything you've read. The wiki keeps getting richer with every source you add and every question you ask.

You never (or rarely) write the wiki yourself — the LLM writes and maintains all of it. You're in charge of sourcing, exploration, and asking the right questions. The LLM does all the grunt work — the summarizing, cross-referencing, filing, and bookkeeping that makes a knowledge base actually useful over time. In practice, I have the LLM agent open on one side and Obsidian open on the other. The LLM makes edits based on our conversation, and I browse the results in real time — following links, checking the graph view, reading the updated pages. Obsidian is the IDE, the LLM is the programmer; the wiki is the codebase.

This can apply to a lot of different contexts. A few examples:

- **Personal** tracking your own goals, health, psychology, self-improvement — filing journal entries, articles, podcast notes, and building up a structured picture of yourself over time.
- **Research** going deep on a topic over weeks or months — reading papers, articles, reports, and incrementally building a comprehensive wiki with an evolving thesis.
- **Reading a book**, filing each chapter as you go, building out pages for characters, themes, plot threads, and how they connect. By the end you have a rich companion wiki. Think of fan wikis like *Tolkien Gateway* — thousands of meticulously pages covering characters, places, events, languages, built by a community of volunteers over years. You could build something like that personally as you read, with the LLM doing all the cross-referencing and maintenance.
- **Business/team**: an internal wiki maintained by LLMs, fed by Slack threads, meeting transcripts, project documents, customer calls. Possibly with humans in the loop reviewing updates. The wiki stays current because the LLM does the maintenance that no one on the team wants to do.
- **Competitive analysis**, due diligence, trip planning, course notes, hobby deep-dives — anything where you're accumulating knowledge over time and want it organized rather than scattered.

항목

지금 방식 — 파일 업로드 · RAG

LLM 위키

자료를 읽는 때

물어볼 때마다 원본을 다시 뒀진다

자료가 들어온 즉시 읽고 정리한다

남는 것

답변 하나 — 대화가 끝나면 사라진다

마크다운 페이지 — 파일로 남는다

자료 사이 연결

물을 때마다 처음부터 새로 엮는다

상호참조가 이미 걸려 있다

어긋나는 자료

부딪쳐도 알아채지 못한다

옛 주장과 부딪치면 표시해 둔다

내가 한 질문

채팅 기록 속에 묻힌다

좋은 답은 새 페이지로 편입된다

시간이 갈수록

처음과 같다

자료를 넣을수록 두꺼워진다

매번 새로 정리하지 않는다 — 한 번 정리해 파일로 남기고, 자료가 늘면 고쳐 쓴다

출처 — gist.github.com/karpathy/442a6bf555914893e9891c11519de94f

위키 한 벌은 세 층으로 되어 있다

층마다 누가 손대는지가 분명히 갈린다. 이 부분이 이 패턴의 거의 전부다.



원본 자료

raw/ 폴더

사람이 직접 고른 논문·
보고서를 그대로 둔다.

누가 손대나

사람이 모은다. LLM은
읽기만 하고 고치지 않는다



위키

wiki/ 폴더

요약·개념·비교 페이지가
링크로 서로 이어진다.

누가 손대나

전부 LLM이 쓴다.
사람은 읽고 방향만 잡는다



규칙

CLAUDE.md

위키를 어떻게 쓰고 관리
할지 적어 둔 파일 하나.

누가 손대나

이게 있어야 챗봇이 아니라
위키 관리자가 된다

특별한 프로그램이 필요 없다. 폴더 하나, 마크다운 파일 여러 개, 규칙을 적은 파일 한 개가 전부다.

index.md(전체 페이지 목록) · log.md(언제 무엇을 넣었는지)를 함께 두는 것을 권한다

실제로 하는 일은 세 가지뿐이다

넣기 · 묻기 · 점검 — 이 셋이 반복되면서 위키가 두꺼워진다

① 넣기 (Ingest) 자료 한 건을 폴더에 떨어뜨린다

LLM이 읽고, 요약 페이지를 쓰고, 관련된 개념 페이지들을 함께 고치고, 기록을 한 줄 남긴다
자료 한 건이 위키 페이지 10~15개를 건드린다 — 처음에는 한 건씩 넣고 요약을 직접 읽어 볼 것

② 묻기 (Query) 원본이 아니라 위키에 묻는다

관련 페이지를 찾아 읽고 출처를 달아 답한다. 표·그래프·슬라이드 등 형식은 그때그때 정한다
핵심은 그다음이다 — 좋은 답은 채팅에 흘러보내지 말고 위키의 새 페이지로 편입시킨다

③ 점검 (Lint) 주기적으로 건강검진을 시킨다

페이지 사이의 모순, 새 자료가 뒤집은 낡은 주장, 아무도 링크하지 않는 고아 페이지를 찾는다
빠진 상호참조와 아직 채우지 못한 빈틈도 짚어 준다 — 다음에 무엇을 더 찾을지가 여기서 나온다

사람이 위키를 포기하는 이유는 유지가 귀찮아서다. LLM은 지겨워하지 않는다.

공개 이후 — 커뮤니티는 어떻게 받아들였나

해커뉴스(Hacker News) 첫날 296점 · 댓글 95개. 환영과 반론이 뚜렷이 갈렸다

반응	대표적인 목소리	새겨 둘 점
환영	관리 비용이 0에 가까워져 위키가 살아남는다	2주 만에 구현체가 쏟아졌다
이건 RAG다	벡터DB만 안 쓸 뿐 결국 검색 보강 아닌가	이름보다 쌓인다는 성질이 요점
새롭지 않다	메모리·스킬로 이미 하던 일 아닌가	차이는 내가 소유한다는 것
인지 아웃소싱	정리하는 그 과정에서 이해가 생기는데	가장 뼈아픈 지적이다
품질 보증	2차 자료를 일일이 검증할 시간이 없다	인용할 숫자는 원본에서 확인
규모의 한계	커지면 사람도 AI도 못 따라가는 지점이 온다	주제를 좁게 잡고 시작한다

쌓이는 것은 위키지 내 이해가 아니다 — 정리는 맡기되, 종합과 판단은 내가 한다

출처 — news.ycombinator.com/item?id=47640875 (2026-04-04) 및 후속 스레드

Level 3

코드 실행

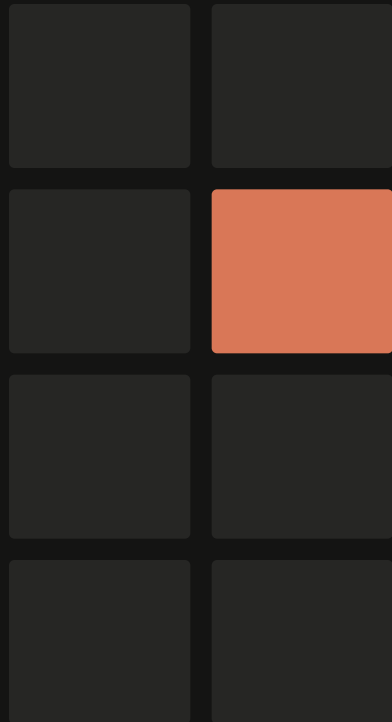
수치모델링 · 컴파일 · 디버깅 · 시각화

Level 1
읽고 쓰기

Level 2
같이 생각하기

Level 3
코드 실행

Level 4
도구 만들기



Demo ⑤ Github의 분석모델 및 데이터 가져오기

Demo ⑤

입력 프롬프트

> GitHub에서 EFDC 소스코드를 받아서,
예제를 실행하고, 결과를 시각화해줘.

사람이 미리 알려준 것
없다. 소스 구조 파악, 빌드 방법, 예제
위치를 도구가 직접 찾아낸다.

- ① 탐색
git clone → 소스 구조 파악 → 빌드 방법 확인
- ② 빌드
Fortran 컴파일(gfortran/ifort)
→ 에러를 자동으로 고친다
- ③ 실행
예제 입력 파일을 찾아 모델을 돌린다
- ④ 시각화
출력 파일을 읽어 matplotlib으로 그린다

처음 보는 모델을 받아서 돌리는 데까지 전 과정이 자동이다. Fortran도 읽는다.

Demo ⑥ 소스코드 분석에서 커스텀 모듈까지

Demo ⑥

입력 프롬프트 — 두 번에 나눠서

> EFDC의 수온 계산 모듈의 로직을 분석해줘.

(설명을 읽고 확인한 뒤)

> 여기에 열교환 항을 추가한 커스텀 모듈을 만들어줘. 먼저 Plan Mode로 설계안을 보여주고, 내가 승인하면 구현해.

① 탐색

Fortran 소스에서 수온 관련 서브루틴을 찾는다

② 분석

호출 관계와 변수 흐름을 따라가며 로직을 설명한다

③ 설계·작성

Plan Mode로 설계한 뒤 Fortran 코드를 쓴다

④ 통합

기존 코드에 넣고 컴파일 테스트, 에러를 고친다

수천 줄 레거시 코드를 읽고 분석해 확장한다. 먼저 설계안을 받아 보는 것이 핵심이다.

복잡하고 장기적인 작업일수록 한땀 한땀

주요 고려 사항

프로젝트 특성에 맞는 별도의 Skill이나 운영규칙이 없는 한, 절대로 한번에 "xxx 해줘!" 금지! 차근차근!

0. Claude.md

프로젝트의 시스템 원칙

1. PRD.md

개발 대상에 대한 스펙 요구사항

Prompt > /grill-me 이 프로젝트의 목적은 xxx를 위해 xx를 분석하기 위한 딥러닝 모델을 개발하는 것이다. 이 프로젝트의 PRD를 작성해줘

2. workflow.md

실제 구현계획 수립

Prompt > PRD의 요구사항을 고려해서 실제 구현계획을 단계별로 수립. 작업요소 간 충돌이 발생하지 않는 범위 내에서 Subagent로 병렬처리를 고려하여 구현계획의 단계적 수립

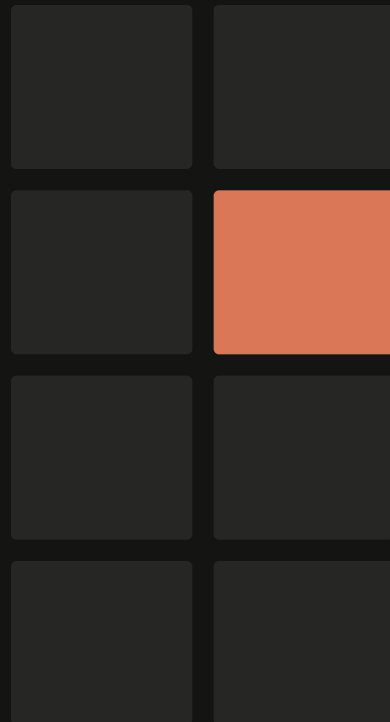
3. 개발 시작

계획 기반 개발작업 진행

Prompt > @workflow.md를 따라 개발 작업을 시작한다. Workflow의 Phase 순서대로 작업을 진행해야하고, 각 Phase를 수행하기 위한 구체적인 TASK를 먼저 계획하고 시작해야 된다. 완료된 항목에 대해서는 log.md 파일을 생성하여 기록

마무리

정리하며



오늘의 핵심 메시지

01 모델의 성능은 하네스가 결정한다

어떤 모델을 쓰느냐보다, 어떤 시스템 안에서 쓰느냐가 결과를 바꾼다.

02 Claude Code로 새로운 무언가를 만들 때는 큰 계획부터 수립하고 세부계획으로 조각해 나간다.

구체적인 실행계획 문서화를 통한 Agent 통제

03 시작은 간단하다

설치 1분, /init 한 번. 일단 시작!

감사합니다

Q & A

질문과 토론을 환영합니다

백 동 해

한국환경연구원 환경평가본부

2026. 08. 27. · AI융복합연구단 월례세미나

